GitHub Actions: Practical CI/CD in Real Repositories

Diogo Ribeiro

February 18, 2026

Contents

Preface

This book is a practical guide to building reliable pipelines with GitHub Actions. It focuses on patterns that survive real repositories: permissions, caching, reuse, testing, and debugging.

Chapter 1

Introduction

GitHub Actions is a CI/CD system driven by events in your repository. A workflow is a YAML file stored in `.github/workflows`. It reacts to events, runs jobs on runners, and produces outputs you can trust.

This book assumes you can read YAML and you know what a CI pipeline should do. The goal is to build automation that is predictable, secure, and easy to change.

1.1 Who this book is for

This book is for engineers who own real repositories:

- teams with multiple services or packages
- projects that need repeatable builds and tests
- repos with a mix of contributors and release cadence

If you are entirely new to CI, read this alongside a short CI primer. If you already use GitHub Actions, use this as a guide to harden and scale workflows.

1.2 What you will build

You will build workflows that cover:

- lint, type-check, and tests on every pull request
- caching to keep builds fast and deterministic
- artifacts for reports and binaries
- composite actions for reuse inside a repo
- reusable workflows for reuse across repos
- secure permissions and secret handling
- release and deployment pipelines that are auditable

1.3 How to read this book

Each chapter introduces a narrow concept and a small, runnable example. Copy the examples into a sandbox repository and adjust them to your stack. Avoid premature complexity; only add features you can explain to a teammate.

Pattern: treat workflows as product code

If a workflow is hard to read, it will be hard to maintain. Keep naming consistent, add comments only where needed, and prefer scripts over inline logic.

1.4 Repository layout

Most examples assume this structure:

- `.github/workflows/` for workflow files
- `scripts/` for repeatable project commands
- `docs/` for generated reports or artifacts

1.5 A first workflow

This example runs on pushes to `main` and on pull requests. It checks out code and runs a script.

```
name: CI (basic)

on:
  push:
    branches: ["main"]
  pull_request:

permissions:
  contents: read

jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Say hello
        run: |
          echo "Hello from GitHub Actions"
```

1.6 A realistic first pipeline

Most teams start with lint and tests, then add cache and artifacts.

```
name: CI (lint + test)

on:
  pull_request:
  push:
    branches: ["main"]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
    - name: Install
      run: ./scripts/install

    - name: Lint
      run: ./scripts/lint

    - name: Test
      run: ./scripts/test
```

1.7 Common early mistakes

- Keeping the workflow logic in YAML instead of scripts.
- Running privileged steps in pull request workflows.
- Ignoring caching until the pipeline is already slow.

1.8 Scope and non-goals

This book focuses on practical workflow design, not on every GitHub Actions feature. We do not cover marketplace action authoring in depth or advanced runner orchestration. Use the patterns here as a foundation, then layer in advanced features as you need them.

1.9 A minimal scripts layout

Move logic out of YAML into small scripts so it can be run locally. Keep the scripts boring and composable.

```
scripts/
  install
  lint
  test
  build
```

Checklist: keep the first workflow boring

- Add `permissions` explicitly.
- Prefer small steps with clear names.
- Keep `run` scripts short; move logic to repo scripts as soon as it grows.
- Treat the workflow as a thin orchestrator, not the place for business logic.

Chapter 2

The mental model

GitHub Actions is built on a small set of concepts. If you understand these, everything else becomes configuration.

2.1 Events and workflows

An *event* happens in the repository: a push, a pull request, a tag, or a schedule. A *workflow* declares which events it listens to and what jobs to run.

Pattern: start from the event

Describe the event first, then design the workflow around it. When the event is clear, jobs and steps follow naturally.

2.2 Jobs and steps

A workflow runs one or more *jobs*. Each job runs on its own runner and is made of sequential *steps*. Jobs run in parallel by default unless you declare dependencies with `needs`.

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/build

  test:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - run: ./scripts/test
```

2.3 Runners

A runner is the machine that executes your job. GitHub-hosted runners are ephemeral; each job starts on a clean VM. Self-hosted runners are long-lived and must be secured and maintained.

2.4 Contexts and expressions

Expressions read data from *contexts* like `github`, `inputs`, or `secrets`. They are written as `${{ ... }}` and evaluated by the workflow engine.

2.5 Environment variables and outputs

Steps can read and write environment variables, and can publish outputs. Outputs are the cleanest way to pass data across steps or jobs.

```
steps:
  - name: Compute version
    id: meta
    run: echo "version=1.2.3" >> "$GITHUB_OUTPUT"

  - name: Use version
    run: echo "Using ${{ steps.meta.outputs.version }}"
```

2.6 Artifacts

Artifacts are files produced by a job that you want to keep. They are stored by GitHub and can be downloaded later. Use artifacts for test reports, build outputs, and deployment packages.

2.7 Concurrency and cancellation

Large repositories need guardrails. Use concurrency groups to avoid running the same workflow twice for the same branch.

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

2.8 Putting it together

Think of a workflow as a thin controller:

- inputs come from events and contexts
- steps call scripts or actions
- outputs are artifacts or job outputs

2.9 State and isolation

Each job starts on a clean machine. Assume nothing persists between jobs unless you use artifacts or caches. This is why moving logic into scripts and storing outputs explicitly matters.

2.10 Event payload as input

The event payload is just data. Treat it like configuration that drives a workflow, not as business logic. If you need branching behavior, prefer small conditional steps over large, hidden logic.

Checklist: mental model

- Events select workflows.
- Workflows run jobs.
- Jobs run steps on runners.
- Outputs connect steps and jobs.
- Artifacts persist files outside the runner.

Chapter 3

YAML and expressions

Workflows are YAML files, but GitHub adds an expression language on top. The combination is powerful and easy to misuse.

3.1 YAML basics that matter

YAML is whitespace-sensitive. Two spaces for indentation is a safe default. Avoid tabs and mixed indentation.

Pattern: keep YAML simple

Avoid anchors and advanced YAML features in workflows. Stick to lists and maps so the workflow stays readable in reviews.

3.2 Expressions

Expressions are written as `${{ ... }}` and evaluated by GitHub. Use them for conditions, values, and dynamic strings.

```
- name: Only on main
  if: ${{ github.ref == 'refs/heads/main' }}
  run: ./scripts/deploy
```

3.3 Common contexts

- `github`: repository, ref, actor, event payload
- `inputs`: workflow or action inputs
- `secrets`: encrypted secrets
- `env`: environment variables
- `steps`: step outputs
- `jobs`: job outputs

3.4 String interpolation

Expressions can be embedded inside strings.

```
- name: Tag image
  run: echo "image:${{ github.sha }}"
```

3.5 Boolean logic

Conditions are evaluated with JavaScript-like semantics. Prefer explicit comparisons over truthy checks.

```
if: ${{ github.event_name == 'pull_request' && github.base_ref == 'main' }}
```

3.6 Functions you will use often

- `contains(list, item)`
- `startsWith(string, prefix)`
- `endsWith(string, suffix)`
- `format(format, ...)`
- `hashFiles(path)`

3.7 Expression pitfalls

- Use quotes around strings inside expressions.
- Use `toJSON` when passing complex values to `run` steps.
- The `env` context is not the same as `secrets`.

Warning: quoting

YAML parsing happens before expression evaluation. If an expression contains `:` or `#`, quote the whole value.

3.8 Readable expressions

Prefer named steps and keep `if` conditions short. When a condition becomes complex, move logic into a script and pass a boolean output.

```
- id: plan
  run: echo "deploy=false" >> "$GITHUB_OUTPUT"

- name: Deploy
  if: ${{ steps.plan.outputs.deploy == 'true' }}
  run: ./scripts/deploy
```

3.9 Defaults and env

Set defaults at the top of the workflow when many jobs share the same values. Use `env` for stable values and inputs for changeable values.

3.10 Expression safety

Avoid relying on implicit type conversions. If a value may be empty, compare against explicit strings. When you pass JSON to a script, use `toJSON` to avoid parsing surprises.

Checklist: expressions

- Prefer explicit comparisons.
- Keep expressions short and local to a step.
- When an expression becomes complex, move logic to a script.

Chapter 4

Triggers and filters

Triggers decide when a workflow runs. Filters decide when it should *not* run.

4.1 Core triggers

- `push`
- `pull_request`
- `workflow_dispatch` (manual)
- `schedule` (cron)
- `workflow_call` (reusable workflows)
- `workflow_run` (after another workflow)

4.2 Branch and path filters

Run only when it matters.

```
on:
  push:
    branches: ["main", "release/*"]
    paths:
      - "src/**"
      - ".github/workflows/**"
    paths-ignore:
      - "docs/**"
```

4.3 Tag filters

Tags are useful for release workflows.

```
on:
  push:
    tags:
      - "v*.*.*"
```

4.4 PR triggers: common pitfalls

- `pull_request` runs on PR events, not on merge commits.
- Use `pull_request_target` carefully; it runs with base repo privileges.
- Avoid running untrusted code with write permissions.

Warning: pull_request_target

It is useful for labeling and triage, but it can expose secrets if you run untrusted code. If you use it, do not checkout and run PR code with write permissions.

4.5 Manual inputs

Manual runs are good for releases and maintenance.

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: "Target environment"
        required: true
        default: "staging"
      dry_run:
        description: "Skip deployment"
        required: false
        default: "true"
```

4.6 Scheduled workflows

Use `schedule` for nightly builds and audits. Remember that schedules are not guaranteed to run at the exact minute.

```
on:
  schedule:
    - cron: "0 2 * * 1-5"
```

4.7 Workflow chaining

Sometimes you want a deployment after tests complete. `workflow_run` provides a safe, explicit chain.

```
on:
  workflow_run:
    workflows: ["CI (lint + test)"]
    types: [completed]
```

4.8 Trigger choice heuristics

Pick the smallest trigger that matches your intent:

- Use `pull_request` for quality gates.
- Use `push` for branch builds and releases.
- Use `workflow_dispatch` for risky operations.

4.9 Event types

Some triggers support multiple event types. If you only care about a subset, declare them explicitly to avoid unexpected runs.

4.10 PR vs push

Use `pull_request` to validate unmerged code and `push` to validate merges. This prevents double work and ensures the merge commit is validated on `main`.

Checklist: triggers

- Be explicit about branches, paths, and tags.
- Use manual inputs for risky operations.
- Prefer `workflow_run` to implicit cross-workflow coupling.

Chapter 5

Jobs and steps

Jobs are the unit of parallelism. Steps are the unit of execution inside a job.

5.1 Job dependencies

Jobs run in parallel by default. Use `needs` to enforce ordering and pass outputs.

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/build

  test:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - run: ./scripts/test
```

5.2 Job outputs

Use outputs to pass data across jobs without recomputation.

```
jobs:
  meta:
    runs-on: ubuntu-latest
    outputs:
      image_tag: ${{ steps.tag.outputs.value }}
    steps:
      - id: tag
        run: echo "value=sha-${GITHUB_SHA}" >> "$GITHUB_OUTPUT"

  deploy:
    runs-on: ubuntu-latest
    needs: meta
    steps:
      - run: echo "Deploying ${{ needs.meta.outputs.image_tag }}"
```

5.3 Steps and shells

Each `run` step uses a shell. On Linux and macOS, it is `bash` by default. On Windows, it is PowerShell. Declare a different shell if you need one.

```
- name: Use bash on Windows
  shell: bash
  run: ./scripts/build
```

5.4 Working directory and environment

Use `working-directory` and `env` to keep commands clean.

```
- name: Run backend tests
  working-directory: services/api
  env:
    NODE_ENV: test
  run: npm test
```

5.5 Services

Service containers are useful for databases or queues. They run alongside your job.

```
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        ports:
          - 5432:5432
    steps:
      - run: ./scripts/test
```

5.6 Timeouts and resilience

Use timeouts to avoid hanging workflows. Use `continue-on-error` only for experimental steps.

```
- name: Optional lints
  continue-on-error: true
  run: ./scripts/lint-extra
```

5.7 Job design heuristics

Good jobs are short and explicit:

- One purpose per job.
- One toolchain per job.
- Prefer artifacts over shared state.

5.8 When to split a job

Split a job when you need isolation, different permissions, or different runners. Keep steps together when they share a working directory and toolchain.

5.9 Retries and flakiness

If a step is flaky, fix the root cause first. If you must retry, do it in a script so the workflow remains simple and explicit.

Checklist: jobs and steps

- Keep jobs focused and composable.
- Use `needs` instead of ad-hoc ordering.
- Prefer outputs over recomputing values.
- Set timeouts for long-running jobs.

Chapter 6

Matrix strategies

Matrix jobs run the same job across multiple configurations. They are ideal for OS, language versions, or test shards.

6.1 A basic matrix

```
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest, windows-latest]
        node: ["18", "20"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
```

6.2 Include and exclude

Use `include` for one-off combinations and `exclude` for invalid ones.

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    python: ["3.11", "3.12"]
    include:
      - os: ubuntu-latest
        python: "3.12"
        experimental: true
    exclude:
      - os: windows-latest
        python: "3.12"
```

6.3 Allowing experimental failures

Mark experimental runs as optional with `continue-on-error`.

```
- name: Run tests
  continue-on-error: ${{ matrix.experimental || false }}
  run: ./scripts/test
```

6.4 Controlling parallelism

Large matrices can overwhelm runners. Use `max-parallel` to limit concurrency.

```
strategy:
  fail-fast: false
  max-parallel: 4
  matrix:
    shard: [1, 2, 3, 4, 5, 6, 7, 8]
```

6.5 Dynamic matrices

You can generate a matrix from a previous step and parse it with `fromJSON`. This keeps the matrix small and relevant.

```
- id: plan
  run: echo 'matrix={"service":["api","web"]}' >> "$GITHUB_OUTPUT"

strategy:
  matrix: ${{ fromJSON(steps.plan.outputs.matrix) }}
```

6.6 Matrix design heuristics

A matrix should represent real product dimensions. Avoid adding variants just because they are easy to add.

- Use matrices for compatibility validation.
- Use shards for speed, not correctness.
- Keep a default job for the most common case.

6.7 Fail-fast tradeoffs

Fail-fast is good for quick feedback but hides the full failure surface. Disable it when you need complete signal across all variants.

6.8 Artifacts per variant

If each variant builds a binary, upload artifacts with the matrix values in the name. This avoids collisions and makes debugging easier.

Checklist: matrix

- Use matrices for the smallest set of meaningful variants.
- Avoid multiplying dimensions without a strong reason.
- Mark experimental variants explicitly.

Chapter 7

Caching

Caching trades storage for speed. A good cache key is specific enough to be correct and broad enough to be reused.

7.1 Cache basics

The cache is keyed by a string and optionally restored by prefix. Caches are scoped to a repository and branch.

```
- name: Cache dependencies
  uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: pip-${{ runner.os }}-${{ hashFiles('**/requirements.txt') }}
    restore-keys: |
      pip-${{ runner.os }}-
```

7.2 What to cache

- Package manager caches (pip, npm, yarn, maven)
- Build tool caches (gradle, bazel, turbo)
- Test data that is expensive to download

7.3 What not to cache

- Secrets or credentials
- Build outputs meant for deployment (use artifacts instead)
- Anything that changes without a reliable key

7.4 Built-in caching

Some setup actions provide caching out of the box. Use them when available.

```
- uses: actions/setup-node@v4
  with:
    node-version: "20"
    cache: "npm"
```

7.5 Cache invalidation

Change the cache key when the dependency graph changes. Use lockfiles to make the key deterministic.

Pattern: hash the lockfile

Hashing `package-lock.json` or `poetry.lock` is a reliable key. Avoid hashing all source files unless you want frequent cache misses.

7.6 Layered caches

Use multiple caches when a toolchain has independent layers. For example, cache both package dependencies and build tool caches separately.

7.7 Measuring cache value

If cache hits do not save meaningful time, remove the cache. Caching adds complexity and can hide problems when dependencies are corrupt.

7.8 Cache scope

Caches are scoped to the repository and branch. Do not assume caches are shared across forks or unrelated branches.

7.9 Warm caches

Run a scheduled workflow to keep caches warm for rarely updated repos. This helps new contributors avoid slow first builds.

Checklist: caching

- Define cache keys with lockfiles.
- Use restore keys to avoid cold starts.
- Cache dependencies, not build outputs.
- Validate that caches actually save time.

Chapter 8

Artifacts

Artifacts are files you want to keep after a job finishes. They are ideal for reports, binaries, and diagnostic outputs.

8.1 Uploading artifacts

```
- name: Upload test report
  uses: actions/upload-artifact@v4
  with:
    name: junit-report
    path: reports/junit.xml
    retention-days: 7
```

8.2 Downloading artifacts

Artifacts can be downloaded by later jobs in the same workflow.

```
- name: Download report
  uses: actions/download-artifact@v4
  with:
    name: junit-report
```

8.3 Artifacts from matrices

Matrix jobs upload multiple artifacts with unique names. Include the matrix value in the artifact name.

```
- name: Upload build
  uses: actions/upload-artifact@v4
  with:
    name: build-${{ matrix.os }}
    path: dist/
```

8.4 Retention and cost

Set retention to the shortest window that still enables debugging. Large artifacts increase storage and download time.

Warning: artifacts are not secrets

Artifacts are stored by GitHub and accessible to anyone with access to the run. Do not upload secrets or credentials.

8.5 Artifact design heuristics

Artifacts should be explicit and minimal:

- name them by purpose and scope
- keep retention short for large files
- avoid uploading the entire workspace

8.6 Artifacts vs releases

Artifacts are temporary and tied to a run. Releases are durable and public-facing. Use artifacts for debugging and staging, and releases for distribution.

8.7 Compression and size

Compress large artifacts before upload when possible. Large files slow down downloads and can degrade developer feedback loops.

Checklist: artifacts

- Upload only what you need to keep.
- Use meaningful names.
- Keep retention short for large files.
- Never store secrets in artifacts.

Chapter 9

Composite actions

Composite actions bundle multiple steps into a reusable action. They live in your repository under `.github/actions/`.

9.1 When to use composite actions

Use them when you repeat the same steps across multiple workflows. If the logic is very specific to one repo, a composite action is lighter than a reusable workflow.

9.2 Action structure

A composite action requires an `action.yml` with inputs and steps.

```
name: "setup tooling"
description: "Install tools used across workflows"
inputs:
  node-version:
    required: true
    default: "20"
runs:
  using: "composite"
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
    - run: npm ci
      shell: bash
```

9.3 Using a composite action

```
- uses: ./.github/actions/setup-tooling
  with:
    node-version: "20"
```

9.4 Inputs and outputs

Composite actions can expose outputs just like workflow steps. Use them to pass versions or paths back to the caller.

```
outputs:
  cache-hit:
    value: ${{ steps.cache.outputs.cache-hit }}
```

9.5 Limitations

- Composite actions cannot define jobs or services.
- They run inside the caller job, so they share its permissions.
- Secrets must be passed as inputs explicitly.

9.6 Versioning

Tag composite actions when they are used across repositories. If they are used only in one repo, keep them on the default branch and version by commit.

9.7 Documentation

Add a short README next to the action with inputs and outputs. This makes it easier to reuse without reading the YAML.

Pattern: keep composite actions small

A composite action should fit on one screen. If it grows into multiple jobs or environments, use a reusable workflow.

Checklist: composite actions

- Make inputs explicit and validated.
- Keep the action deterministic.
- Prefer a composite action over copy-paste.

Chapter 10

Reusable workflows

Reusable workflows allow you to share pipelines across repositories. They are defined like normal workflows but triggered with `workflow_call`.

10.1 Defining a reusable workflow

```
name: Reusable CI

on:
  workflow_call:
    inputs:
      node-version:
        required: true
        type: string
    secrets:
      npm_token:
        required: true

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ inputs.node-version }}
      - run: npm ci
      - run: npm test
        env:
          NPM_TOKEN: ${{ secrets.npm_token }}
```

10.2 Calling a reusable workflow

```
name: CI

on:
  pull_request:
  push:
    branches: ["main"]

jobs:
```

```
ci:
  uses: org/shared-workflows/.github/workflows/node-ci.yml@v1
  with:
    node-version: "20"
  secrets:
    npm_token: ${{ secrets.NPM_TOKEN }}
```

10.3 Inputs and defaults

Treat inputs like an API. Provide defaults for common cases and keep optional inputs minimal.

10.4 Permissions and inheritance

Reusable workflows do not automatically inherit permissions. Set permissions in the caller or inside the reusable workflow explicitly.

Pattern: lock versions

Reference reusable workflows by a tag or commit SHA. This makes upgrades explicit and auditable.

10.5 Testing shared workflows

Create a small sandbox repository that calls the reusable workflow. This keeps changes verifiable before a broad rollout.

10.6 Input validation

Validate inputs early in the workflow and fail fast with clear messages. Do not allow a workflow to proceed with an invalid environment or missing secret.

10.7 Compatibility

If many repos depend on a shared workflow, keep changes backward compatible. Introduce new inputs with defaults and deprecate old ones gradually.

Checklist: reusable workflows

- Keep inputs and secrets small and explicit.
- Define permissions deliberately.
- Version shared workflows and document changes.

Chapter 11

Versioning and releases

Release automation is only reliable if your versioning strategy is clear. Decide how versions are produced and how artifacts are published.

11.1 Semantic versioning

Use `MAJOR.MINOR.PATCH` where:

- MAJOR breaks compatibility
- MINOR adds features
- PATCH fixes bugs

11.2 Tag-driven releases

Tagging a commit is the simplest way to trigger a release pipeline.

```
name: Release

on:
  push:
    tags:
      - "v*.*.*"

permissions:
  contents: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ./scripts/build
      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/
```

11.3 Release branches

For long-lived products, you may need maintenance branches. Keep release workflows identical across branches and use tags to signal releases.

11.4 Changelogs

Generate changelogs as part of the release. Use commit conventions or labels to make automation reliable.

11.5 Manual releases

A manual release is still automation. Use `workflow_dispatch` inputs to select version, environment, and dry-run.

```
on:
  workflow_dispatch:
    inputs:
      version:
        description: "Release version"
        required: true
      dry_run:
        description: "Skip publish"
        required: false
        default: "true"
```

11.6 Release promotion

Promote the same artifact from staging to production instead of rebuilding. This keeps releases reproducible and easier to audit.

11.7 Signing and provenance

If you ship binaries, consider signing them. Use attestations or provenance to prove where and how the build was produced.

Pattern: release is a workflow, not a human checklist

If you can ship by running `workflow_dispatch` and selecting a version, you can ship on Friday with confidence.

Checklist: releases

- Tag releases with a consistent prefix.
- Store build outputs as artifacts.
- Keep release scripts in `scripts/`.

Chapter 12

Permissions and secrets

Security starts with the `permissions` block. If you do not set it, GitHub applies defaults that may be too broad.

12.1 The GITHUB_TOKEN

Every workflow receives a `GITHUB_TOKEN`. Set permissions at the workflow or job level.

```
permissions:
  contents: read
  pull-requests: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

12.2 Job-level permissions

You can narrow permissions for specific jobs. This is useful when a single job needs write access.

```
jobs:
  publish:
    permissions:
      contents: write
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/publish
```

12.3 Secrets

Secrets are encrypted values stored in GitHub. Use them via the `secrets` context.

```
- name: Deploy
  env:
    API_TOKEN: ${{ secrets.API_TOKEN }}
  run: ./scripts/deploy
```

12.4 Environment protection

Use environments to add approvals and separate secrets for staging and production. The environment name can be set per job.

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: ./scripts/deploy
```

12.5 Least privilege patterns

Start with `contents: read` and add only what you need. Separate risky jobs into a dedicated workflow when possible.

12.6 Forked pull requests

Secrets are not exposed to workflows triggered by forks by default. Design your PR workflows so they can run without secrets.

12.7 Secret rotation

Rotate secrets regularly and automate the rollout. Use environment-scoped secrets so production values stay isolated.

Warning: secrets in logs

Do not echo secrets. GitHub masks secrets, but it is safer to avoid printing them entirely.

Checklist: permissions and secrets

- Define `permissions` explicitly.
- Use job-level permissions for write access.
- Scope secrets to environments.
- Avoid running privileged jobs on untrusted PRs.

Chapter 13

OIDC and cloud authentication

OpenID Connect (OIDC) allows workflows to get short-lived cloud credentials. This removes the need for long-lived access keys in secrets.

13.1 How OIDC works

GitHub issues an identity token to the workflow. Your cloud provider validates that token and issues temporary credentials.

13.2 Required permissions

You must allow the `id-token` permission in your workflow.

```
permissions:
  contents: read
  id-token: write
```

13.3 Example: AWS

This example exchanges the OIDC token for AWS credentials.

```
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789012:role/ci-deploy
    aws-region: us-east-1
```

13.4 Example: Azure

```
- uses: azure/login@v2
  with:
    client-id: ${{ secrets.AZURE_CLIENT_ID }}
    tenant-id: ${{ secrets.AZURE_TENANT_ID }}
    subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```

13.5 Example: GCP

```
- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider: ${{ secrets.GCP_WORKLOAD_PROVIDER }}
    service_account: ci-deploy@project.iam.gserviceaccount.com
```

13.6 Trust policies

OIDC only works if the cloud trust policy restricts which workflows can assume a role. Always pin repository, branch, and workflow identifiers.

13.7 Audience and claims

Cloud providers often validate the token audience and claims. Use the default audience unless your provider requires a custom value.

13.8 Debugging OIDC

If authentication fails, log the provider response and verify the trust policy. Most issues come from mismatched repository or branch claims.

Pattern: prefer OIDC for deployments

If your cloud provider supports OIDC, use it. It reduces secret sprawl and limits credential lifetime.

Checklist: OIDC

- Enable `id-token: write` only where needed.
- Use short-lived roles with narrow permissions.
- Audit trust policies regularly.

Chapter 14

Supply chain security

Your workflow depends on actions and packages you did not write. Supply chain security is about minimizing that risk.

14.1 Pin actions

Use tags for convenience, but prefer commit SHAs for sensitive workflows.

```
- uses: actions/checkout@v4
# safer but more maintenance:
# - uses: actions/checkout@<commit-sha>
```

14.2 Limit permissions

Most workflows only need `contents: read`. Give write permissions to a single job, not the whole workflow.

14.3 Dependency scanning

Automate updates with tools like Dependabot. Treat action updates like dependency updates: review and test them.

14.4 Restrict network access

Self-hosted runners should use network policies and secrets isolation. Avoid running untrusted pull requests on self-hosted runners.

14.5 Provenance and attestations

For critical releases, generate provenance or build attestations. This improves traceability and strengthens the supply chain story.

14.6 Action allowlists

If your organization supports action allowlists, use them. Restricting which actions can run reduces the attack surface.

14.7 Update cadence

Pin actions, but keep them updated. Schedule dependency updates and review them like code changes.

Warning: third-party actions

Every external action is a new dependency. Only use actions you trust, and keep them updated.

Checklist: supply chain

- Pin critical actions.
- Minimize permissions and secrets exposure.
- Separate untrusted PRs from privileged workflows.

Chapter 15

Debugging workflows

Debugging CI is about making the failure reproducible and visible. Start by reducing the workflow to the smallest failing case.

15.1 Use step logs

Logs are the primary debugging tool. Group logs to keep output readable.

```
- name: Build
  run: |
    echo "::group::Build output"
    ./scripts/build
    echo "::endgroup::"
```

15.2 Enable debug logging

GitHub supports two debug flags. Set them as repository secrets or workflow env vars.

```
env:
  ACTIONS_STEP_DEBUG: true
  ACTIONS_RUNNER_DEBUG: true
```

15.3 Rerun jobs

Rerun failed jobs from the Actions UI. This saves time when failures are flaky or transient.

15.4 Collect diagnostics

When a failure is intermittent, capture extra artifacts.

```
- name: Upload logs
  if: ${{ failure() }}
  uses: actions/upload-artifact@v4
  with:
    name: debug-logs
    path: logs/
```

15.5 Local reproduction

Reproduce failures locally with the same script the workflow runs. If you cannot run it locally, move more logic into scripts until you can.

15.6 Workflow isolation

Create a minimal workflow that runs only the failing job. This reduces noise and isolates the root cause faster.

15.7 Manual debug runs

Add a `workflow_dispatch` entry to run a workflow with specific inputs. This is useful when you need to debug with a particular branch or parameter set.

15.8 Artifacts on failure

If a job fails intermittently, upload logs or state only on failure. This keeps successful runs clean while preserving enough detail to investigate.

Checklist: debugging

- Make the failing step minimal.
- Increase logging only when needed.
- Capture diagnostics on failure.
- Reproduce locally with the same scripts.

Chapter 16

Performance and scale

As repositories grow, workflows become slower and more expensive. Performance work is about removing unnecessary work and controlling concurrency.

16.1 Cancel redundant runs

Use concurrency to cancel obsolete runs when new commits arrive.

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

16.2 Reduce the work

- Use path filters to skip workflows when nothing relevant changed.
- Split expensive tests into shards.
- Cache dependencies aggressively.

16.3 Parallelize safely

Matrix jobs increase parallelism but also cost. Limit with `max-parallel` and avoid large cartesian products.

16.4 Self-hosted runners

Self-hosted runners can be faster and cheaper for heavy workloads. They also require patching, monitoring, and security hardening.

16.5 Workflow reuse

Reuse workflows instead of duplicating logic across repos. Centralized workflows are easier to optimize.

16.6 Measuring performance

Track total pipeline time, queue time, and failure rate. Optimize the slowest 20 percent of jobs first.

16.7 Queue time

Long queue times often indicate limited runners or oversized matrices. Reduce the matrix size or add runners before optimizing job steps.

16.8 Cost controls

Use path filters, job timeouts, and max-parallel limits to control costs. Review which workflows run on every push and reduce unnecessary triggers.

Checklist: performance

- Cancel redundant runs.
- Filter by paths and branches.
- Cache what is expensive to rebuild.
- Limit matrix size.
- Measure and prioritize the slowest jobs.

Appendix A

Expression reference

This appendix lists the most common contexts and functions. It is not exhaustive, but it is enough for everyday workflows.

A.1 Contexts

- `github`: repo, ref, actor, event payload
- `env`: environment variables
- `inputs`: inputs to workflows and actions
- `secrets`: encrypted secrets
- `steps`: step outputs and conclusions
- `jobs`: job outputs
- `runner`: runner metadata
- `matrix`: matrix values
- `strategy`: strategy configuration

A.2 Useful functions

- `success()` / `failure()` / `cancelled()`
- `always()`
- `contains(a, b)`
- `startsWith(a, b)` / `endsWith(a, b)`
- `format(fmt, ...)`
- `join(list, sep)`
- `toJSON(obj)` / `fromJSON(str)`
- `hashFiles(path)`

A.3 Common string patterns

Use `format` for building readable strings:

```
run: echo "${{ format('build-{0}-{1}', github.sha, runner.os) }}"
```

A.4 Condition examples

```
if: ${{ github.event_name == 'pull_request' }}
if: ${{ startsWith(github.ref, 'refs/tags/v') }}
if: ${{ failure() && github.ref == 'refs/heads/main' }}
```

```
if: ${{ always() && (cancelled() || failure()) }}
```

A.5 Matrix access

Matrix values are accessed through `matrix`.

```
- run: echo "Running on ${{ matrix.os }}"
```

A.6 Job outputs

Job outputs are available through `needs`.

```
- run: echo "Artifact is ${{ needs.build.outputs.artifact_name }}"
```

A.7 Troubleshooting matrix

Symptom	Likely cause	Fix
Workflow does not run	Trigger does not match branch or paths	Check `on` filters and branch names
Step skipped	`if` condition evaluated to false	Log the inputs and simplify conditions
Secret is empty	Secret not set in repo or environment	Verify secret name and environment scope
Cache miss	Key changed or restore key too narrow	Use lockfile hash and a broad restore key

A.8 Permission reference

Permission	Typical use
`contents: read`	Checkout code and read repository data
`contents: write`	Create releases or push tags
`pull-requests: write`	Comment or label pull requests
`id-token: write`	OIDC authentication with cloud providers

A.9 Debugging expressions

When conditions behave unexpectedly, log the values you compare. Prefer printing explicit context values instead of entire payloads.

Checklist: expressions

- Keep conditions short.
- Avoid complex logic in YAML.
- Prefer scripts for heavy logic.

Appendix B

Reference templates

These are minimal templates you can copy into real repositories. Keep them small and move logic into `scripts/`.

B.1 Python CI template

```yaml
name: CI (python)

on:
  pull_request:
  push:
    branches: ["main"]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12"]

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Test
        run: |
          pytest -q
```

B.2 Node + Yarn CI template

```yaml
name: CI (node)

on:
  pull_request:
  push:
    branches: ["main"]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "yarn"

      - name: Install
        run: yarn install --frozen-lockfile

      - name: Test
        run: yarn test
```

B.3 Release on tags template

```yaml
name: Release

on:
  push:
    tags:
      - "v*.*.*"

permissions:
  contents: write

jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build artifact
        run: |
          mkdir -p dist
          echo "example" > dist/example.txt
      - name: Upload
        uses: actions/upload-artifact@v4
        with:
          name: release-dist
          path: dist/
```

B.4 Reusable workflow template

```
name: Shared CI

on:
  workflow_call:
    inputs:
      node-version:
        required: true
        type: string
    secrets:
      npm_token:
        required: true

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ inputs.node-version }}
      - run: npm ci
      - run: npm test
        env:
          NPM_TOKEN: ${{ secrets.npm_token }}
```

B.5 Docker build template

```
name: Build container

on:
  push:
    branches: ["main"]

permissions:
  contents: read
  packages: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: |
          docker build -t ghcr.io/org/app:${GITHUB_SHA} .
      - name: Push image
        run: |
          echo ${{ secrets.GITHUB_TOKEN }} | docker login ghcr.io -u ${{ github.actor
              }} --password-stdin
          docker push ghcr.io/org/app:${GITHUB_SHA}
```

B.6 Scheduled maintenance template

```
name: Nightly maintenance

on:
  schedule:
    - cron: "0 3 * * *"

permissions:
  contents: read

jobs:
  cleanup:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ./scripts/cleanup
```

B.7 Matrix test template

```
name: CI (matrix)

on:
  pull_request:

permissions:
  contents: read

jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, macos-latest]
        node: ["18", "20"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
```

B.8 Monorepo path-filter template

```
name: CI (monorepo)

on:
  pull_request:
    paths:
      - "services/api/**"
      - ".github/workflows/**"

permissions:
  contents: read
```

```
jobs:
  api:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ./services/api/scripts/test
```

Closing note

If you treat workflows as code, you will naturally build better automation. Keep examples runnable, keep permissions minimal, and keep reuse explicit.